

Lab: 2D Convolution — AXI-Stream & AXI-Lite (Fixed-Point)

Verified against `design_1_fixed.bit` / `design_1_fixed.hwh` .

Parameter	Value
HLS data type	<code>ap_fixed<16, 3></code> (Q3.13)
Integer bits	3 $\rightarrow$ range <code>[-4, +3.99988]</code>
Fractional bits	13 $\rightarrow$ scale = 8192
IP instance	<code>conv2d_stream_0</code>
DMA instance	<code>axi_dma_0</code>
CTRL offset	<code>0x00</code>
rows offset	<code>0x10</code>
cols offset	<code>0x18</code>
kernel[0] offset	<code>0x20</code> (each element +4 bytes)

1. Load Overlay

```
In [12]: import time
import numpy as np
import matplotlib.pyplot as plt
from pynq import Overlay, allocate
from scipy.signal import convolve2d
from PIL import Image

# Fixed-point Q3.13 parameters
SCALE = 8192 # 2^13

def float_to_q(f):
    """Converts a Python float to a 16-bit Q3.13 integer."""
    return int(round(f * SCALE)) & 0xFFFF

ol = Overlay("design_1_fixed.bit")
print("Overlay loaded:", list(ol.ip_dict.keys()))

dma = ol.axi_dma_0
conv2d = ol.conv2d_stream_0
```

```
Overlay loaded: ['conv2d_stream_0', 'axi_timer_0', 'axi_dma_0', 'processing_s
ystem7_0']
```

2. Prepare Test Image & Kernel

Key constraint: `ap_fixed<16,3>` has integer range `[-4, +4]`. An unscaled edge-detect kernel (centre = 8) applied to a `[0,1]` image produces outputs up to **8.0** → **overflow**. We scale the kernel by **0.25** so max output = 2.0, safely inside range.

`MAX_WIDTH = 128` in HLS, and the DMA limit $\approx 16\,383$ bytes → use **63 × 64 = 4 032 pixels × 4 B = 16 128 B**.

```
In [13]: # --- NEW CODE ---
from PIL import Image

ROWS = 64
COLS = 64

# 1. Load the image and convert to Grayscale ("L")
img = Image.open("cat.jpg").convert("L")

# 2. Resize to fit DMA limits.
# Note: PIL .resize() expects (width, height) which is (COLS, ROWS)
img_resized = img.resize((COLS, ROWS))

# 3. Convert to numpy array and normalize to [0.0, 1.0]
# CRITICAL: If you don't divide by 255, values like 150 will
# instantly overflow your ap_fixed<16,3> range of [-4, +4]!
image_in = np.array(img_resized, dtype=np.float32) / 255.0

print(f"Loaded image shape: {image_in.shape}")
print(f"Image value range : [{image_in.min():.3f}, {image_in.max():.3f}]")

# 3x3 Edge Detection Kernel (scaled by 0.25 to prevent Q3.13 overflow)
KERNEL_SCALE = 0.25
kernel_3x3 = KERNEL_SCALE * np.array([
    [-1, -1, -1],
    [-1,  8, -1],
    [-1, -1, -1]
], dtype=np.float32)

print(f"Image size: {ROWS}x{COLS}")
```

```
Loaded image shape: (64, 64)
Image value range : [0.106, 0.965]
Image size: 64x64
```

3. Configure IP via AXI-Lite

Register map (from `design_1_fixed.hwh`):

Register	Offset
CTRL	0x00
rows	0x10
cols	0x18

Register	Offset
kernel[0]	0x20
kernel[i]	0x20 + i×4

```
In [14]: CTRL_REG      = 0x00
ROWS_OFFSET = 0x10
COLS_OFFSET = 0x18
KERNEL_BASE = 0x20 # Verified offset for fixed-point design

# Write image dimensions
conv2d.write(ROWS_OFFSET, ROWS)
conv2d.write(COLS_OFFSET, COLS)

# Write 3x3 Kernel (Packing two 16-bit values per 32-bit AXI word)
kernel_flat = kernel_3x3.flatten()
for i in range(0, 9, 2):
    val_low = float_to_q(kernel_flat[i])
    val_high = float_to_q(kernel_flat[i+1]) if i+1 < 9 else 0

    packed_32 = (val_high << 16) | val_low
    conv2d.write(KERNEL_BASE + (i // 2) * 4, packed_32)
```

4. DMA Transfer

```
In [15]: n_pixels = ROWS * COLS
in_buf = allocate(shape=(n_pixels,), dtype=np.int32)
out_buf = allocate(shape=(n_pixels,), dtype=np.int32)

# Vectorized quantization: Multiply by SCALE and cast to int32
in_buf[:] = np.round(image_in.flatten() * SCALE).astype(np.int32)
out_buf[:] = 0

t0 = time.perf_counter()

# DMA order: Setup receiver BEFORE sending
dma.recvchannel.transfer(out_buf)
dma.sendchannel.transfer(in_buf)

# Start IP
conv2d.write(CTRL_REG, 0x01)

# Wait for completion
dma.sendchannel.wait()
dma.recvchannel.wait()

t_dma = time.perf_counter() - t0
print(f"Processed {n_pixels} pixels in {t_dma*1e3:.2f} ms")

# Dequantize: cast to int16 (to restore sign), cast to float, divide by SCALE
raw_out = np.array(out_buf, dtype=np.int32)
image_out_hw = (raw_out.astype(np.int16).astype(np.float32) / SCALE).reshape((

Processed 4096 pixels in 3.75 ms
```

5. Dequantise & Verify

```

In [16]: # Golden software model
image_out_sw = convolve2d(image_in, kernel_3x3, mode='same', boundary='fill',

# Compare valid inner window [1:-1, 1:-1]
hw_valid = image_out_hw[1:-1, 1:-1]
sw_valid = image_out_sw[1:-1, 1:-1]

max_diff = np.max(np.abs(hw_valid - sw_valid))
tolerance = 10.0 / SCALE # 10 LSBs of acceptable quantization noise

print(f"Max |HW - SW| (inner window) = {max_diff:.5f}")
if max_diff <= tolerance:
    print("PASS: Hardware output matches Software within fixed-point bounds!")
else:
    print("WARNING: Outputs differ beyond acceptable tolerance.")

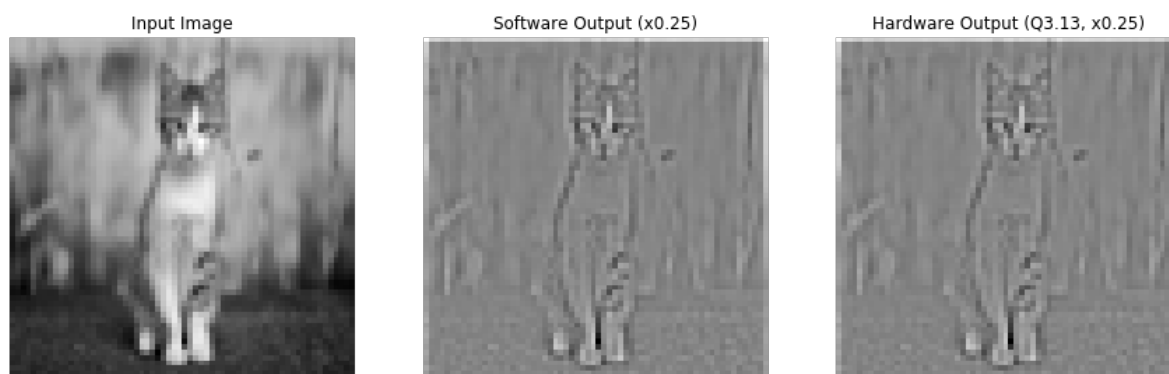
# Visualization
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(image_in, cmap='gray')
axes[0].set_title("Input Image")
axes[1].imshow(image_out_sw, cmap='gray')
axes[1].set_title(f"Software Output (x{KERNEL_SCALE})")
axes[2].imshow(image_out_hw, cmap='gray')
axes[2].set_title(f"Hardware Output (Q3.13, x{KERNEL_SCALE})")

for ax in axes:
    ax.axis('off')
plt.show()

```

Max |HW - SW| (inner window) = 0.00025

PASS: Hardware output matches Software within fixed-point bounds!



```

In [11]: in_buf.freebuffer()
         out_buf.freebuffer()

```

```

In [ ]:

```

